

Heapsort Analysis

Time Complexity Analysis

The running time of Heapsort comes from two major operations:

1. Building the max heap.
2. Repeatedly removing the maximum element from the heap.

Step 1: Building the Max-Heap

The heap is created by calling `heapify()` on all non-leaf nodes.

The number of nodes requiring heapification is approximately:

$$n/2$$

Each heapify operation takes at most:

$$O(\log n)$$

However, not every node requires the full height adjustment. The total cost of building a heap is:

$$O(n)$$

Therefore:

Heap Construction Complexity = $O(n)$

Step 2: Extracting Maximum Elements

After building the heap, the algorithm removes the maximum element one by one.

For each extraction:

- The root element is swapped with the last element.
- The heap property is restored using `heapify()`.

Each heapify operation requires:

$$O(\log n)$$

Since this operation is performed for every element:

$$n * O(\log n)$$

Therefore: **$O(n \log n)$**

Worst-Case Time Complexity

In the worst case, every extraction requires the heap to be completely adjusted.

The total cost is: **$O(n) + O(n \log n)$**

Ignoring lower-order terms: **$O(n \log n)$**

The worst case occurs regardless of input ordering because Heapsort always builds the heap and performs the same number of extraction operations.

Average-Case Time Complexity

Unlike Quicksort, Heapsort does not depend on the initial arrangement of elements.

For any input distribution:

- The heap construction remains $O(n)$.
- Each of the n removals requires $O(\log n)$.

Therefore: **$O(n \log n)$**

The average case is also: **$O(n \log n)$**

Best-Case Time Complexity

Even if the input array is already sorted:

- The algorithm still creates a max heap.
- It still performs n extraction operations.

Therefore, the best-case complexity is **$O(n) + O(n \log n)$**

which becomes: **$O(n \log n)$**

Why Heapsort is $O(n \log n)$ in All Cases

Heapsort has a consistent runtime because it does not make decisions based on the input order.

Unlike Quicksort, where a poor pivot selection can produce unbalanced partitions, Heapsort always performs:

1. A linear-time heap construction:

$$O(n)$$

2. Exactly n heap removals:

$$n * O(\log n)$$

Therefore:

$$O(n) + O(n \log n) = O(n \log n)$$

This makes Heapsort a reliable sorting algorithm with guaranteed performance.

Space Complexity Analysis

Heapsort is an **in-place sorting algorithm** because it does not require additional arrays or data structures.

The algorithm only uses a few variables:

- Index variables
- Temporary variables for swapping elements

Therefore:

$$O(1)$$

auxiliary space complexity.

The recursive implementation of `heapify()` requires stack memory:

$$O(\log n)$$

because the recursion depth is equal to the height of the heap.

An iterative version of `heapify` can reduce this overhead and achieve true: **$O(1)$** extra space

Priority Queue Implementation Analysis

Data Structure

Choice of Data Structure

A **Python list** is used to represent the binary heap. The list provides an efficient way to store a complete binary tree because the parent-child relationships can be calculated directly using array indices without requiring explicit pointers.

For a node stored at index i :

- Parent index:
 - $(i-1)/2$
- Left child:
 $2i+1$
- Right child:
 $2i+2$

Using a list offers several advantages:

- Simple implementation.
- Efficient memory utilization.
- Constant-time access to any element.
- Efficient insertion and deletion while maintaining the heap property.

Because heap operations involve frequent swapping of elements, using a list minimizes implementation complexity and provides good overall performance.

Task Representation

Each task is represented using a Task class containing the following information:

- **task_id** – Unique identifier for the task.
- **priority** – Priority value used for scheduling.
- **arrival_time** – Time at which the task enters the system.
- **deadline** – Time by which the task should ideally complete.

Choice of Heap

A **max heap** is used for the priority queue.

In a max heap, the task with the **highest priority** is always stored at the root and can be removed in constant time after heap adjustment.

This approach is suitable for scheduling systems where higher-priority tasks must always be executed before lower-priority tasks.

Time Complexity Analysis

1. insert(task)

The insert() operation first places the new task at the end of the heap and then repeatedly compares it with its parent. If the new task has a higher priority, the two elements are swapped until the heap property is restored. In the worst case, the new task may move from a leaf node to the root of the heap. Since the height of a binary heap containing n elements is $O(\log n)$, the insertion operation has a worst-case time complexity of: $O(\log n)$

2. extract_max()

The extract_max() operation removes the root element, which contains the highest-priority task, replaces it with the last element in the heap, and then restores the heap property by moving the new root downward. In the worst case, the element travels from the root to a leaf, resulting in a time complexity of: $O(\log n)$

3. increase_key(task, new_priority)

The implementation first searches for the task using its task_id, which requires scanning the heap. This search takes $O(n)$ time. After updating the priority, the task may move upward through the heap, which takes $O(\log n)$ time. Therefore, the overall worst-case complexity is dominated by the search: $O(n)$

If the position of each task were maintained using an auxiliary hash table, the search could be performed in constant time, reducing the overall complexity of the operation to $O(\log n)$.

4. is_empty()

The is_empty() operation simply checks whether the heap contains any elements by evaluating the length of the underlying list. This operation requires constant time: $O(1)$

Summary of Time Complexities

Operation	Time Complexity
Insert	$O(\log n)$
Extract Maximum	$O(\log n)$
Increase Key*	$O(n)$
Is Empty	$O(1)$

The provided implementation has $O(n)$ complexity because it performs a linear search to locate the task. With an additional hash map that stores the index of each task in the heap, this operation can be improved to $O(\log n)$ after constant-time lookup.